



Introducción a PL/pgSQL

Programación Procedimental en PostgreSQL

Base de Datos Avanzadas

UP Chiapas · Mtro. Ramsés Camas



github.com/RamsesCamas/BDA_Slides_And_Code

Evaluación del Curso

50%

Defensa oral individual

Demostrar competencia en vivo: escribir y explicar código SQL/PL sin ayuda

25%

Proyecto final con defensa

Proyecto integrador con preguntas individuales a cada miembro del equipo

15%

Quizzes en clase (pizarrón/oral)

Preguntas al azar al inicio de clase — sin dispositivos, sin aviso previo

10%

Labs entregados

Ejercicios prácticos de laboratorio entregados a tiempo



Mínimo 60 en defensa oral o se reprueba automáticamente, sin importar los demás componentes.

Agenda de Hoy



¿Qué es PL/pgSQL y para qué sirve?

15 min



Bloques DO: tu primer código procedimental

15 min



Variables y tipos de datos

15 min



Control de flujo: IF / ELSE / ELSIF

20 min



Loops: LOOP, WHILE, FOR

20 min



Funciones: CREATE FUNCTION

20 min



Ejercicio práctico

15 min

Total: ~2 horas · Requisito: Docker Compose del repositorio levantado

¿Qué es PL/pgSQL?



PL/pgSQL = Procedural Language / PostgreSQL. Es un lenguaje de programación que vive DENTRO de la base de datos y te permite escribir lógica con variables, condiciones y loops.



Analogía: SQL normal es como dar órdenes sueltas. PL/pgSQL es como escribir un programa completo que la BD ejecuta por ti.

SQL puro:

```
SELECT nombre FROM empleados
WHERE salario > 10000;
```

PL/pgSQL:

```
DO $$
DECLARE
    total INTEGER;
BEGIN
    SELECT COUNT(*) INTO total
    FROM empleados;
    RAISE NOTICE 'Hay % empleados', total;
END $$;
```

¿Para Qué Sirve PL/pgSQL?



Funciones reutilizables

Lógica que se ejecuta muchas veces con diferentes datos



Validaciones complejas

Reglas de negocio que un CHECK simple no puede expresar



Triggers (próxima clase)

Código que se ejecuta automáticamente ante eventos



Stored Procedures

Operaciones complejas en una sola llamada desde la app



Automatización interna

Calcular campos, limpiar datos, generar reportes



En la industria, PL/pgSQL se usa en el 80% de las bases PostgreSQL en producción.

Tu Primer Bloque: DO

El bloque DO es código anónimo — se ejecuta una vez y no se guarda. Perfecto para practicar.

```
DO $$
```

```
DECLARE
```

```
-- aquí van las variables
```

```
BEGIN
```

```
-- aquí va tu código
```

```
END
```

```
$$;
```

\$\$ marca el inicio y fin del bloque. Es el delimitador.

DECLARE es opcional. Solo si necesitas variables.

BEGIN...END encierra toda tu lógica. Es obligatorio.

El ; al final ejecuta todo el bloque de una vez.

RAISE NOTICE: Tu print() en SQL

PL/pgSQL no tiene pantalla, así que usamos RAISE NOTICE para imprimir mensajes en la consola.

Básico:

```
DO $$ BEGIN
  RAISE NOTICE 'Hola desde PL/pgSQL!';
END $$;
```

Con variables (% es el placeholder):

```
DO $$
DECLARE
  nombre TEXT := 'Ramsés';
  edad INTEGER :=
24;
BEGIN
  RAISE NOTICE 'Hola %, tienes % años', nombre, edad;
END $$;
```

OUTPUT: NOTICE: Hola Ramsés, tienes 24 años

Variables en PL/pgSQL

Se declaran en la sección DECLARE, antes de BEGIN.

```
nombre_variable TIPO [ := valor_inicial ];
```

Tipo	Descripción	Ejemplo
INTEGER	Números enteros	edad INTEGER := 25;
NUMERIC	Decimales precisos	precio NUMERIC := 99.50;
TEXT	Texto sin límite	nombre TEXT := 'Ana';
VARCHAR(n)	Texto con límite	codigo VARCHAR(10);
BOOLEAN	true / false	activo BOOLEAN := true;
DATE	Fecha	hoy DATE := CURRENT_DATE;
RECORD	Fila completa	fila RECORD;



:= es el operador de asignación en PL/pgSQL (NO el = de SQL)

SELECT INTO: Guardar Resultados

Para meter el resultado de una query en una variable, usamos SELECT ... INTO.

Un solo valor:

```
DO $$
DECLARE
    total INTEGER;
BEGIN
    SELECT COUNT(*) INTO total FROM empleados;
    RAISE NOTICE 'Total: %', total;
END $$;
```

Múltiples columnas:

```
DO $$
DECLARE
    v_nombre TEXT;
    v_salario NUMERIC;
BEGIN
    SELECT nombre, salario INTO v_nombre, v_salario
    FROM empleados WHERE id = 1;
    RAISE NOTICE '=: $%', v_nombre, v_salario;
END $$;
```



¡A Practicar!

1. Levanta tu Docker Compose y entra a psql
2. Crea un bloque DO que imprima tu nombre y tu edad
3. Crea un bloque DO que cuente cuántas tablas tienes y lo imprima
4. Crea un bloque que guarde el MAX(salario) en una variable e imprima el resultado

5 minutos 

Control de Flujo: IF / ELSE

Sintaxis:

```
IF condición THEN
    -- código si true

ELSIF otra_condición THEN
    -- código alternativo

ELSE
    -- si nada se cumple

END IF;
```

Ejemplo:

```
DO $$
DECLARE
    salario NUMERIC;
BEGIN
    SELECT salario INTO salario
    FROM empleados WHERE id = 1;

    IF salario > 50000 THEN
        RAISE NOTICE 'Salario alto';
    ELSIF salario > 25000 THEN
        RAISE NOTICE 'Salario medio';
    ELSE
        RAISE NOTICE 'Salario bajo';
    END IF;
END $$;
```



Ojo: Es ELSIF (sin la segunda E), no ELSEIF. Y siempre cierra con END IF; (con espacio).

CASE: Switch/Case de PL/pgSQL

Cuando tienes muchas condiciones sobre la misma variable, CASE es más limpio que muchos ELSIF.

```
DO $$
DECLARE
    depto TEXT;
    mensaje TEXT;
BEGIN
    SELECT departamento INTO depto
    FROM empleados WHERE id = 1;

    CASE depto
        WHEN 'Ventas' THEN mensaje := 'Área comercial';
        WHEN 'IT' THEN mensaje := 'Área técnica';
        ELSE mensaje := 'Otro departamento';
    END CASE;

    RAISE NOTICE '%', mensaje;
END $$;
```

Loops: LOOP Básico

LOOP se repite infinitamente hasta que tú lo detengas con EXIT.

```
DO $$  
DECLARE  
    contador INTEGER := 1;  
BEGIN  
    LOOP  
        RAISE NOTICE 'Iteración: %', contador;  
        contador := contador + 1;  
        EXIT WHEN contador > 5;  
    END LOOP;  
END $$;
```

Salida:

```
NOTICE: Iteración: 1  
NOTICE: Iteración: 2  
NOTICE: Iteración: 3  
NOTICE: Iteración: 4  
NOTICE: Iteración: 5
```

WHILE: Loop con Condición

Más intuitivo que LOOP + EXIT. Se repite MIENTRAS la condición sea verdadera.

```
DO $$  
DECLARE  
    i INTEGER := 1;  
BEGIN  
    WHILE i <= 5 LOOP  
        RAISE NOTICE 'Número: %', i;  
        i := i + 1;  
    END LOOP;  
END $$;
```

WHILE

Úsalo cuando sabes la condición de parada pero no el número exacto de iteraciones.

FOR (siguiente slide)

Úsalo cuando sabes exactamente cuántas veces iterar o necesitas recorrer un query.

FOR: El Loop Más Útil

FOR numérico (como un for de Python/JS):

```
DO $$ BEGIN
  FOR i IN 1..5 LOOP
    RAISE NOTICE '%', i;
  END LOOP;
END $$;
```



La variable `i` se crea automáticamente — no necesitas declararla en `DECLARE`. Solo existe dentro del loop.

FOR sobre un query (recorrer filas):

```
DO $$
DECLARE
  fila RECORD; -- variable tipo fila
BEGIN
  FOR fila IN SELECT nombre, salario FROM empleados LOOP
    RAISE NOTICE '% gana %', fila.nombre, fila.salario;
  END LOOP;
END $$;
```



¡Practiquemos IF y Loops!

1. Escribe un DO que clasifique un salario como alto/medio/bajo usando IF
2. Escribe un WHILE que imprima los números del 10 al 1 (cuenta regresiva)
3. Escribe un FOR que recorra todos los empleados e imprima nombre + departamento
4. Bonus: Combina FOR + IF para imprimir solo empleados con salario > 30000

CREATE FUNCTION: Código Reutilizable

Un bloque DO se ejecuta y desaparece. Una FUNCTION se guarda en la BD y se puede llamar muchas veces.

```
-- Crear la función
CREATE OR REPLACE FUNCTION saludo(nombre TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN 'Hola, ' || nombre || '!';
END;
$$ LANGUAGE plpgsql;

-- Llamar la función
SELECT saludo('María');
-- Resultado: 'Hola, María!'
```

Función con Lógica Real

Calcular el bono de un empleado según su departamento y antigüedad.

```
CREATE OR REPLACE FUNCTION calcular_bono(emp_id INTEGER)
RETURNS NUMERIC AS $$
DECLARE
    v_salario NUMERIC;
    v_depto TEXT;
    v_bono NUMERIC;
BEGIN
    SELECT salario, departamento
    INTO v_salario, v_depto
    FROM empleados WHERE id = emp_id;

    CASE v_depto
        WHEN 'Ventas' THEN v_bono := v_salario * 0.15;
        WHEN 'IT'      THEN v_bono := v_salario * 0.10;
        ELSE           v_bono := v_salario * 0.05;
    END CASE;

    RETURN v_bono;
END; $$ LANGUAGE plpgsql;
```

Uso: `SELECT nombre, calcular_bono(id) FROM empleados;` · Combina variables, SELECT INTO, CASE y RETURN.

RETURNS VOID y PERFORM

No todas las funciones devuelven algo. A veces solo ejecutan una acción.

Función que no retorna valor:

```
CREATE OR REPLACE FUNCTION limpiar_inactivos()
RETURNS VOID AS $$
BEGIN
    DELETE FROM empleados WHERE activo = false;
    RAISE NOTICE 'Empleados inactivos eliminados';
END; $$ LANGUAGE plpgsql;

-- Para llamar funciones VOID, usa SELECT:
SELECT limpiar_inactivos();
```

PERFORM: ejecutar SELECT sin guardar resultado

```
-- Dentro de PL/pgSQL, si haces un SELECT sin INTO,
-- PostgreSQL da error. Usa PERFORM en su lugar:

PERFORM pg_sleep(1); -- espera 1 segundo
```

Resumen: Lo Que Aprendimos Hoy

`DO $$ ... $$;`

Bloques anónimos para código rápido

`DECLARE`

Sección para declarar variables con tipos

`RAISE NOTICE`

Imprimir mensajes (tu print/console.log)

`SELECT INTO`

Guardar resultados de queries en variables

`IF / ELSIF / ELSE`

Condiciones y bifurcaciones

`CASE`

Switch/case para múltiples condiciones

`LOOP / WHILE / FOR`

Repetir código (FOR es el más útil)

`CREATE FUNCTION`

Código guardado y reutilizable

Próxima clase: Triggers y Stored Procedures (necesitas esto como base)



Ejercicio Final

Crea una función clasificar_empleado(emp_id INTEGER) que:

1. Reciba el ID de un empleado
2. Consulte su salario y departamento con SELECT INTO
3. Use IF o CASE para clasificarlo: 'Junior' (< 25k), 'Mid' (25k-50k), 'Senior' (> 50k)
4. Retorne un TEXT con: 'Juan - IT - Senior (\$65,000)'
5. Bonus: Usa un FOR para llamar la función para TODOS los empleados

